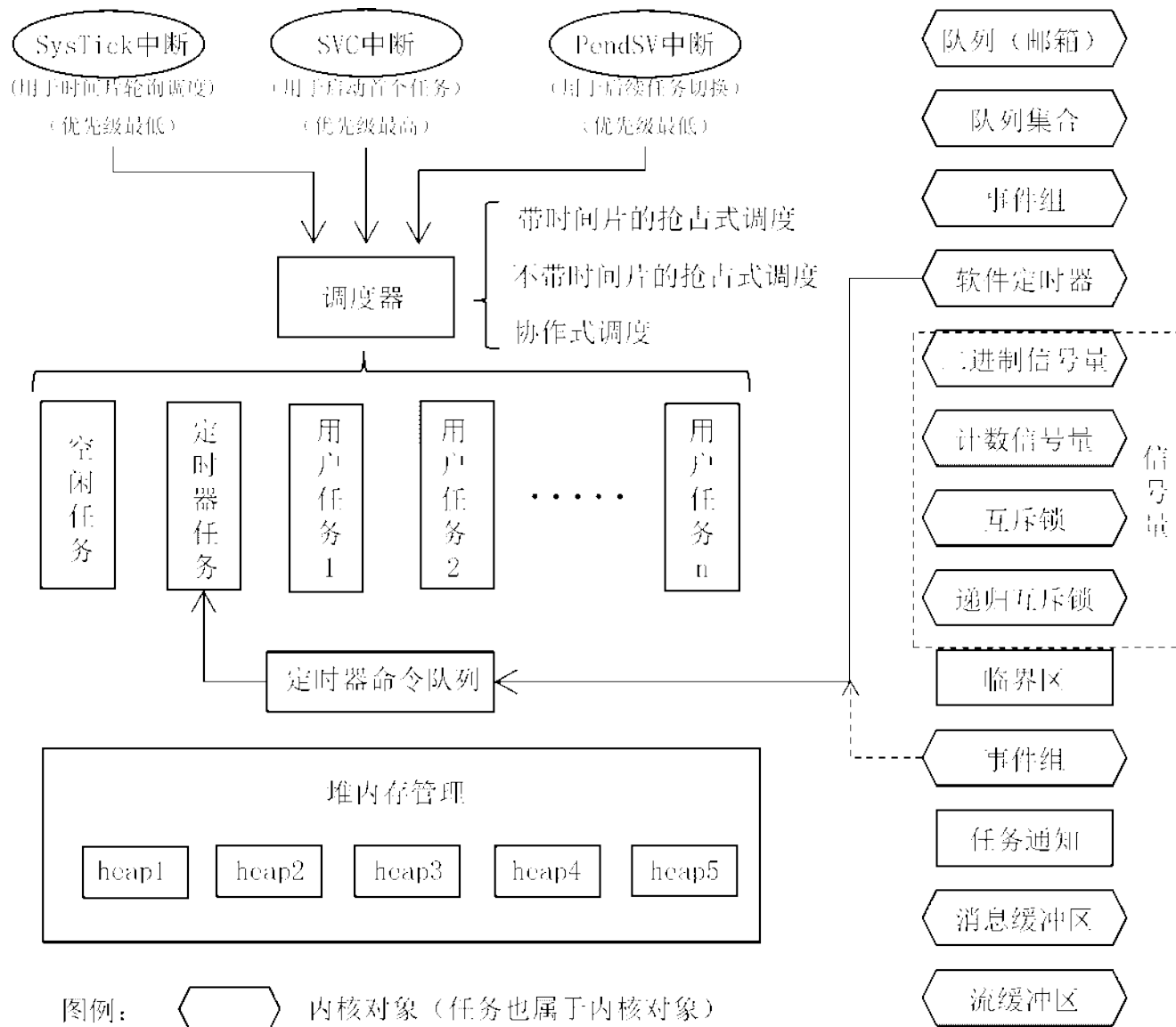


FreeRTOS总图

2026年9月21日 16:17



堆栈管理

2026年9月21日 16:35

Cortex-M3内核相当于电脑的CPU，所有的指令和计算都在内核里执行；Flash相当于电脑的硬盘，程序和常量存储在它里边；SRAM相当于电脑的内存，所有变量都存储在SRAM当中

SRAM里一共有4个段，即.data段、.bss段、.heap段和.stack段，另外.heap和.stack之间还有一段未使用的区域。对于这4个段来说，每个段存储不同的内容。比如.data段用来存储带有初值的全局变量和静态变量；.bss段用来存储无初值（或者零初值）的全局变量和静态变量；.heap段用来存储堆变量，调用malloc()分配的内存就是从.heap段里“拿”出来的；.stack段用来存储“栈”，局部变量等就是被保存在栈里的

类型	变量名	存储位置
未初始化的全局变量/静态变量	g_globalVar, g_globalArray	.bss段
有初值的全局变量/静态变量	g_staticGlobalVar, num	.data段
局部变量	i, n, ret, pHeapVar指针本身	栈(.stack段)
参数	input	栈(.stack段)
malloc()分配的变量	pHeapVar指向的内存	堆(.heap段)

内存碎片化是指经过多次动态内存分配和释放后，空闲内存被分割成许多不连续的小块，导致即使总空闲内存足够，也无法满足一个较大的连续内存分配请求的现象

堆内存管理：

head_1---只分配不释放

优势：简单、高效、速度快。它的工作原理非常简单，执行起来速度会非常快，非常适合用在实时性要求高的场合。另外由于heap_1只分配、不释放，因此不会产生碎片化问题。如果程序里只创建内核对象而不删除，那么用heap_1是非常合适的。

head_2---最佳匹配算法

使用最佳匹配算法（Best Fit Algorithm），会优先选择满足要求的最小空闲块不能合并相邻的空闲块，会造成严重的内存碎片化问题（这是heap_2被淘汰的主要原因）

head_3---改进标准C库

标准C库的malloc()和free()不适合用在基于FreeRTOS的代码里。两个原因，一个是执行时间

长且不确定，另一个是不可重入。

缺点：耗时，且执行时间不确定

// Heap_3里的pvPortMalloc()的实现

```
void *pvPortMalloc(size_t xWantedSize)
{
    vTaskSuspendAll(); //暂停调度器，阻止其它任务执行
    pvReturn = malloc(xWantedSize); // 调用标准C库里的malloc()方法实现内存分配
    xTaskResumeAll(); // 恢复调度器
    return pvReturn;
}
```

// Heap_3里的vPortFree()的实现

```
void vPortFree( void * pv )
{
    vTaskSuspendAll(); // 暂停调度器，阻止其它任务执行
    free( pv ); // 调用标准C库里的free()方法实现内存释放
    xTaskResumeAll(); // 恢复调度器
}
```

head_4---合并相邻空闲块（最佳）

heap_4是在heap_2的基础上改进得来的。和heap_2一样，heap_4也是使用了“最佳匹配算法”，但区别在于heap_4现在能够合并相邻空闲块了，这极大地减少了heap_2带来的内存碎片化问题

head_5---能够使用不连续内存

任务管理

2026年9月20日 11:54

简介：该方法用于创建任务

返回：pdPASS - 成功；

errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY - 内存不足以创建任务

```
BaseType_t xTaskCreate(TaskFunction_t pvTaskCode, /* 任务代码 */
                      const char *const pcName, /* 字符串名称，调试时供人类阅读 */ 在Config可配置最大字符长度
                      unsigned short usStackDepth, /* 任务栈的深度 */
                      void *pvParameters, /* 任务参数 */
                      UBaseType_t uxPriority, /* 优先级，0 ~ configMAX_PRIORITY-1 */
                      TaskHandle_t *pxCreatedTask /* 输出参数，用来返回任务句柄 */
);
```

创建一个任务所需要的SRAM空间分成两个部分：即“TCB”和“任务栈”。TCB是Task Control Block的缩写，翻译成中文叫做任务控制块，它里边存储了跟任务运行相关的信息

	FreeRTOS里的 任务优先级	stm32f1单片机里的 中断优先级
作用对象	任务	中断
执行机构	FreeRTOS里的调度器	Cortex-M3内核里的NVIC模块
含义	表示任务执行的优先顺序	表示中断响应的优先顺序
范围	0 ~ configMAX_PRIORITY	0 ~ 15，有分组
规则	数字越大，优先级越高	数字越小，优先级越高

任务状态

运行（Running）状态，是指某个任务正在被CPU执行

就绪（Ready）状态，是指某个任务正在等待被CPU执行

阻塞（Blocked）状态，是指某个任务因为自身没有满足“先决条件”，“自愿放弃”被执行的机会

所谓暂停（Suspended）状态，就是指某个任务被禁止使用CPU

FreeRTOS的调度算法总共有3种：

带时间片的抢占式调度（Preemptive Scheduling with Time Slicing）

不带时间片的抢占式调度（Preemptive Scheduling without Time Slicing）

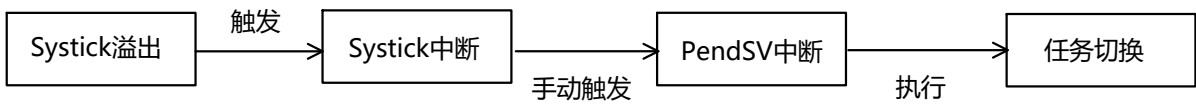
协作式调度（Co-operative Scheduling）

抢占式与协作式区别：抢占式高优先级任务推出阻塞状态后**直接打断**正在进行的低优先级任务，
协调式高优先级任务推出阻塞后需**等待**正在进行的低优先级任务**执行完成后**才能执行

时间片底层逻辑：**相同优先级的任务轮番执行，以保证每个任务享有均等的执行时间**

configUSE_PREEMPTION - 是否启用抢占。0 - 禁用抢占；1 - 启用抢占。

configUSE_TIME_SLICING - 是否使用时间片。0 - 禁用时间片； 1- 启用时间片



二进制信号量

2026年9月21日 20:01

```
#include "Semaphr.h"
```

```
// 作用：创建二进制信号量
```

```
// 返回：新创建的二进制信号量的句柄，创建失败时返回NULL
```

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

```
// 作用：删除信号量，比如二进制信号量、计数信号量、互斥锁、递归互斥锁
```

```
// 参数：xSemaphore - 要被删除的信号量的句柄
```

```
void vSemaphoreDelete(SemaphoreHandle_t xSemaphore);
```

```
// 作用：给信号。适用于所有的信号量类型。
```

```
// 参数：xSemaphore - 信号量的句柄
```

```
// 返回：pdPASS - 成功，pdFAIL - 失败
```

```
BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore);
```

```
// 作用：取信号。适用于所有信号量。
```

```
// 参数：xSemaphore - 信号量的句柄
```

```
// 参数：xTicksToWait - 最长等待时间，portMAX_DELAY表示无限等待
```

```
// 返回：pdPASS - 成功取到信号；pdFAIL - 因超时而失败
```

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);
```

二进制信号量在中断的用法

```
void xxx_IRQHandler(void)
```

```
{
```

```
    // 1. 在开头声明xHigherPriorityTaskWoken变量
```

```
    // 2. 一定要将初始值设置为pdFALSE
```

```
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
```

```
    // 3. 可以多次调用FromISR版本的API，以&xHigherPriorityTaskWoken作为参数
```

```
    xxxFromISR(, &xHigherPriorityTaskWoken);
```

```
    xxxFromISR(, &xHigherPriorityTaskWoken);
```

```
    ...
```

```
    xxxFromISR(, &xHigherPriorityTaskWoken);
```

```
    // 4. 调用portYIELD_FROM_ISR()进行可能的任务切换
```

```
    // 把xHigherPriorityTaskWoken作为参数
```

```
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
```

```
}
```

计数信号量

2026年9月22日 15:17

```
#include "FreeRTOS.h"
#include "semphr.h"
```

API调用

// @作用：创建计数信号量

// @参数：xMaxCount - 计数信号量所能够达到的最大计数值，超过这个值之后将无法继续给信号

// @参数：uxInitialCount - 计数信号量被创建时所持有的计数值

// @返回：创建成功后将返回计数信号量的句柄，如果因为堆内存不足而创建失败则返回NULL

//

```
SemaphoreHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount, UBaseType_t uxInitialCount);
```

// 作用：删除信号量，比如二进制信号量、计数信号量、互斥锁、递归互斥锁

// 参数：xSemaphore - 要被删除的信号量的句柄

```
void vSemaphoreDelete(SemaphoreHandle_t xSemaphore);
```

// 作用：给信号。适用于所有的信号量类型。

// 参数：xSemaphore - 信号量的句柄

// 返回：pdPASS - 成功，pdFAIL - 失败

```
BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore);
```

// 作用：给信号，可以使用在中断上下文当中。

// 参数：xSemaphore - 信号量的句柄

// 参数：pxHigherPriorityTaskWoken - 输出参数，如果调用该API使得更高优先级的任务退出阻塞则该参数输出pdTRUE

// 返回：pdPASS - 成功，pdFAIL - 失败

```
BaseType_t xSemaphoreGiveFromISR( SemaphoreHandle_t xSemaphore, BaseType_t *pxHigherPriorityTaskWoken);
```

// 作用：获取信号量的当前计数值

// 参数：xSemaphore - 信号量的句柄

// 返回：信号量的计数值，对于二进制信号量只会返回0或1

```
UBaseType_t uxSemaphoreGetCount(SemaphoreHandle_t xSemaphore);
```

// 作用：取信号。适用于所有信号量。

// 参数：xSemaphore - 信号量的句柄

// 参数：xTicksToWait - 最长等待时间，portMAX_DELAY表示无限等待

// 返回：pdPASS - 成功取到信号；pdFAIL - 因超时而失败

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);
```

两个用法：

1.事件计数

2.守护资源

队列

2026年9月22日 15:43

API调用

```
#include "FreeRTOS.h"
```

```
#include "queue.h"
```

```
//
```

```
// @作用: 创建队列
```

```
// @参数: uxQueueLength - 队列的长度
```

```
// @参数: uxItemSize - 队列当中每一个元素的大小, 单位是字节
```

```
// @返回: 队列创建成功时返回新创建的队列的句柄, 失败时返回NULL
```

```
//
```

```
QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t  
uxItemSize);
```

```
//
```

```
// @作用: 删除队列
```

```
// @参数: pxQueueToDelete - 要删除的队列的句柄
```

```
//
```

```
void vQueueDelete(QueueHandle_t pxQueueToDelete);
```

```
//
```

```
// @作用: 向队列发送数据
```

```
// @参数: xQueue - 要操作的队列的句柄
```

```
// @参数: pvItemToQueue - 指向要发送的数据的指针
```

```
// @参数: xTicksToWait - 超时时间, 即队列满时等待的最长时间, portMAX_DELAY表示无限  
长。如果超时后队列仍无空位则发送失败。
```

```
// @返回: pdPASS - 发送成功; pdFAIL - 发送失败 (超时)
```

```
//
```

```
BaseType_t xQueueSend(QueueHandle_t xQueue, const void * pvItemToQueue,  
TickType_t xTicksToWait);
```

[xQueueSendFromISR\(\) - xQueueSend\(\)的FromISR版本](#)

```
//
```

```
// @作用: 从中断上下文里向队列发送数据, 是xQueueSend()的FromISR版本
```

```
// @参数: pvItemToQueue - 指向要发送的数据的指针
```

```
// @参数: pxHigherPriorityTaskWoken - 输出参数, 如果调用该API能够使更高优先级的任务退  
出阻塞
```

```
//
```

则该参数会输出pdTRUE

```
// @返回: 如果发送数据成功则返回pdPASS, 如果队列满则返回pdFAIL
```

```
//
```

```
BaseType_t xQueueSendFromISR( QueueHandle_t xQueue,
```

```

const void *pvItemToQueue,
BaseType_t *pxHigherPriorityTaskWoken);

//
// @作用: 从队列里接收数据
// @参数: xQueue - 队列的句柄
// @参数: pvBuffer - 接收缓冲区, 用于存放从队列里读出的数据
// @参数: xTicksToWait - 超时时间, 当队列为空的时候再接收数据, 任务会进入阻塞
//
// 该参数用来设置最大阻塞时间
// @返回: 读取成功时返回pdPASS, 如果队列空且读取超时则返回pdFAIL
//
BaseType_t xQueueReceive(QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);

xQueueReceiveFromISR() - xQueueReceive()的FromISR版本
//
// @作用: xQueueReceive()的FromISR版本
// @参数: xQueue - 队列的句柄
// @参数: pvBuffer - 接收缓冲区, 用于存放从队列里边取出的数据
// @参数: pxHigherPriorityTaskWoken - 输出参数, 如果调用该API能够使更高优先级的任务退出阻塞
//
// 则该参数会输出pdTRUE
// @返回: 如果接收数据成功则返回pdPASS, 如果队列空则返回pdFAIL
//
BaseType_t xQueueReceiveFromISR( QueueHandle_t xQueue,
                                void *pvBuffer,
                                BaseType_t *pxHigherPriorityTaskWoken);

```

邮箱

邮箱并不是FreeRTOS里的一种内核对象, 它只是队列的一种特殊用法而已。确切的说, 创建队列的时候将uxQueueLength设置为1就相当于创建了一个邮箱

API调用:

```

#include "FreeRTOS.h"
#include "queue.h"

```

```

//
// @作用: 向队列发送数据, 如果已有数据则会覆写
// @参数: xQueue - 队列的句柄
// @参数: pvItemToQueue - 指向要发送的数据的指针
// @返回: 覆盖操作不会失败, 因此总是返回pdPASS
// @注意! 这个API仅能用在长度等于1的队列里
//
BaseType_t xQueueOverwrite(QueueHandle_t xQueue, const void *pvItemToQueue);

//
// @作用: 从队列当中读一个项目, 但不把它取出

```

```
// @参数: xQueue - 队列的句柄
// @参数: pvBuffer - 缓冲区, 用来接收读到的数据
// @参数: xTicksToWait - 超时时间, 如果队列为空则会等待, 该参数用于设置最长等待时间
// @返回: 读取成功时返回 pdPASS, 超时返回pdFAIL
//
BaseType_t xQueuePeek( QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);
```

软件定时器

2026年9月23日 14:20

两个主要功能:

- 1. 定时执行代码
- 2. 周期性地执行代码

配置文件:

```
#define configUSE_TIMERS 1 // 启用软件定时器功能

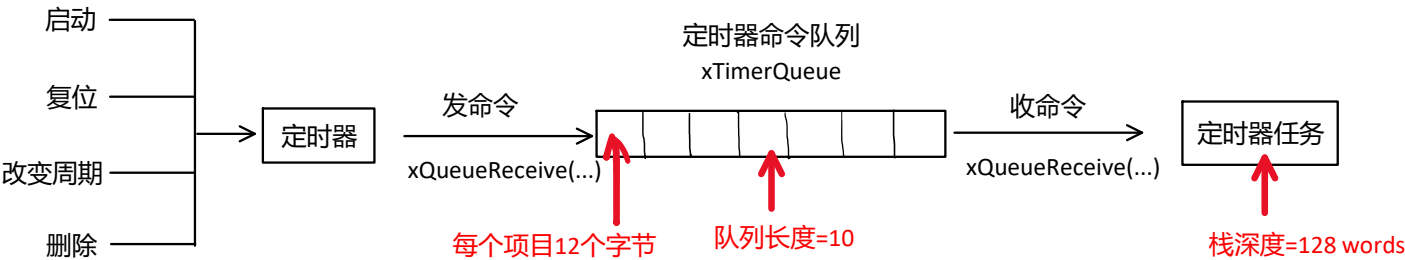
#define configTIMER_QUEUE_LENGTH 10 // 定时器命令队列长度10

#define configMAX_PRIORITIES 5 // 任务优先级数量

#define configTIMER_TASK_PRIORITY ( configMAX_PRIORITIES - 1 ) // 定时器任务取最高优先级4

#define configMINIMAL_STACK_SIZE 128 // 最小栈深度128

#define configTIMER_TASK_STACK_DEPTH configMINIMAL_STACK_SIZE // 定时器任务取最小栈深度128
```



API调用:

```
#include "FreeRTOS.h"
#include "timers.h"

//
// @作用: 创建一个软件定时器
// @返回: 成功, 则返回新创建的软件定时器定时器的句柄; 失败, 则返回NULL
//
TimerHandle_t xTimerCreate( const char *pcTimerName, // 定时器的绰号, 供调试工具使用, 只给人看、对代码运行无影响
                             const TickType_t xTimerPeriod, // 周期, 间隔多久才会执行代码, 单位: Ticks
                             const UBaseType_t uxAutoReload, // 是否自动重装, 即: 只做一次还是重复执行。
                             void * const pvTimerID, // 定时器的ID, 类似任务参数, 当多个定时器共用回调函数时使用。不用时填NULL
                             TimerCallbackFunction_t pxCallbackFunction); // 回调函数, 定时器到期后要执行的代码

//
```

```

// @作用：启动软件定时器
// @参数：xTimer - 软件定时器的句柄
// @参数：xTicksToWait - 定时器命令队列满的时候最长等待的时间
// @返回：pdPASS - 成功；pdFAIL - 因超时失败
//
BaseType_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait);

//
// @作用：停止定时器
// @参数：xTimer - 定时器的句柄
// @参数：xTicksToWait - 当定时器命令队列满之后最长的阻塞时间
// @返回：pdPASS - 成功，pdFAIL - 因超时而失败
//
BaseType_t xTimerStop(TimerHandle_t xTimer, TickType_t xTicksToWait);

//
// @作用：改变软件定时器的周期（即到期时间）
// @参数：xTimer - 定时器的句柄
// @参数：xNewPeriod - 新的周期值
// @参数：xTicksToWait - 定时器命令队列满之后，最长的阻塞时间
// @返回：成功 - pdPASS；因超时而失败 - pdFAIL
//
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer, TickType_t xNewPeriod, TickType_t
xTicksToWait);

//
// @作用：对定时器进行复位操作，让定时器重新计算到期时间
// @参数：xTimer - 定时器的句柄
// @参数：xTicksToWait - 定时器命令队列满之后，最长的阻塞时间
// @返回：成功 - pdPASS；因超时而失败 - pdFAIL
//
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait);

//
// @作用：删除定时器，释放资源
// @参数：xTimer - 定时器的句柄
// @参数：xTicksToWait - 队列满后的最长阻塞时间
// @返回：成功 - pdPASS，因超时而失败 - pdFAIL
//
BaseType_t xTimerDelete( TimerHandle_t xTimer, TickType_t xTicksToWait);

```

事件组

2026年9月20日 11:53

EventBits_t 高八位为控制字节 剩下位宽为事件比特 (0 1)

修改EventBits_t位宽—FreeRTOSConfig.h修改#define configTICK_TYPE_WIDTH_IN_BITS配置项

#define configUSE_EVENT_GROUPS 0 配置文件改为1才能使能事件组

EventGroupHandle_t xEventGroupCreate(void); 创建事件组

void vEventGroupDelete(EventGroupHandle_t xEventGroup); 删除事件组

EventBits_t xEventGroupGetBits(EventGroupHandle_t xEventGroup); 读取事件组值

EventBits_t xEventGroupSetBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToSet); 向事件组写1, 此API有返回值, 为写完后的事件组值

EventBits_t xEventGroupClearBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear); 向事件组写0

xEventGroupWaitBits()

// @作用: 等待事件组的比特位置位

// @返回: 执行清除操作之前的值 (如果xClearOnExit == pdTRUE的话会执行清除操作)

```
EventBits_t xEventGroupWaitBits ( const EventGroupHandle_t xEventGroup, // 事件组的句柄
                                const EventBits_t uxBitsToWaitFor,      // 掩码 表明要等待的位
                                const BaseType_t xClearOnExit,           // 等待操作成功后是否执行清除操作
                                // pdFALSE - 不执行清除操作
                                // pdTRUE - 等待成功后执行清除操作
                                const BaseType_t xWaitForAllBits,        // 是否等到所有比特位置位
                                // pdFALSE - 只需等待任一比特位
                                // pdTRUE - 需要等待所有比特位
                                TickType_t xTicksToWait );               // 超时时间
```

相比于创建一组二进制信号量, 创建事件组会消耗更少的内存

访问公共资源

2026年9月24日 15:00

竞态条件

在单片机的世界里有很多可以共享的资源，这些资源里边蕴藏着危险。当多个任务同时访问某个共享资源的时候，程序的执行结果将变得不确定，其结果如何将取决于任务代码的执行顺序。我们很难预料哪个任务的某条代码先执行，哪个任务的某条代码后执行，因此这种状态下程序的执行结果将变得不确定。我们可以使用一个专业术语来描述这种混乱，它叫做“竞态条件”。

“竞态条件”的源头来自共享资源，当我们使用单片机编程的时候，这种共享资源主要有三类：共享的变量、不可重入的函数以及共享的外设。

共享的变量：全局变量、静态局部变量

外设（Peripheral）：指的是除单片机内核以外的硬件资源。它可以是单片机上的片上外设，比如GPIO、USART、I2C等，也可以是单片机之外的硬件资源，比如外接的LCD显示器、蓝牙模块、wifi模块等。与全局变量或静态局部变量一样，这些硬件资源同样是可以被多个任务共享的，也同样会引发“竞态条件”

不可重入的函数：**可重入性是指，当一个函数尚未返回时，再次调用它，两次调用之间不会相互干扰并能够输出正确的结果。**一般来说，如果一个函数使用了共享的资源，而又不加任何防护措施，那么它会变成“不可重入”的

“竞态条件”会让程序的执行结果变得不确定，甚至出现严重的错误，因此我们要想办法避免“竞态条件”。在这方面FreeRTOS已经给我们提供了很多可用的技术手段，这些手段包括了：暂停调度器、临界代码段、互斥锁、递归互斥锁、守门员任务、邮箱等等。本节课我们先介绍第一种，也就是“暂停调度器”。

暂停调度器：把FreeRTOS的调度器关掉，从而禁止任何形式的任务切换

FreeRTOS提供了两个API，分别用来暂停和恢复调度器。其中vTaskSuspendAll()用来暂停调度器，xTaskResumeAll()用来恢复调度器

// @作用：暂停调度器，期间禁止任何任务切换

```
void vTaskSuspendAll(void);
```

// @作用：恢复调度器

// @返回：恢复调度器期间是否执行了被挂起的切换，pdTRUE - 是，pdFALSE - 否。

```
BaseType_t xTaskResumeAll(void);
```

暂停调度器的负面效果是“影响系统的实时性”

暂停调度器适合保护极短的代码，长时间暂停调度器会严重破坏系统的实时性。

临界区：是通过屏蔽中断来实现的。临界区的作用是屏蔽中断，所以进入临界区之后PendSV中断也会被屏蔽，这样也能达到暂停任务切换的效果

并不会屏蔽所有的中断，而是只会屏蔽优先级不高于configMAX_SYSCALL_PRIORITY的中断

互斥锁

```
#include "FreeRTOS.h"
```

```
#include "semaphore.h"
```

// @作用：创建互斥锁

// @返回：创建成功后返回新建的互斥锁的句柄，如果失败则返回NULL

```
SemaphoreHandle_t xSemaphoreCreateMutex(void);
```

```

// @作用：删除互斥锁
// @参数：xSemaphore - 被删除的互斥锁的句柄
void vSemaphoreDelete( SemaphoreHandle_t xSemaphore );

// @作用：获取锁（其实就是取信号）
// @参数：xSemaphore - 互斥锁的句柄
// @参数：xTicksToWait - 未获取到锁时的最长等待时间
// @返回：pdPASS - 成功获取到锁；pdFAIL - 未获取到锁
BaseType_t xSemaphoreTake( SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait );

// @作用：释放锁（其实就是给信号）
// @参数：xSemaphore - 互斥锁的句柄
// @返回：pdPASS - 给信号成功，pdFAIL - 当前任务并未持有锁，因此释放锁失败
BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore);

```

要保护长代码的话推荐使用互斥锁，因为它只会让竞争使用资源的任务进入阻塞，而不会做出类似关闭调度器或者屏蔽中断这种极端的动作。在持有锁期间，高优先级的任务和中断能够正常抢占，实时性不受影响

互斥锁也有其自身的短板：

1. 首先，互斥锁是一种内核对象，创建它要消耗相当的系统资源。
2. 互斥锁的执行速度稍慢于暂停调度器，如果要保护的代码非常短的话，还是推荐使用暂停调度器或者临界区。

守门员任务：队列

按键

2026年9月21日 16:35

按键驱动

由于每隔IO都有上拉电阻，将按钮对应的端口设置为GPIO输入上拉模式，按钮按下时为低电平，只需捕捉IO电平上升沿就是按钮按下松开的时刻

捕捉上升沿方式：在一个for循环，每间隔一段时间读取IO引脚的值，如果出现相邻两个值第一次是1，第二次是0，那就表示我们捕捉到了上升沿。

按钮复用：

- 1.代码封装，将零散代码写成函数，编写驱动层为适配后边多个按钮
- 2.定义结构体为后续各个按钮作为使用驱动函数的传参

```
typedef struct
{
    GPIO_PinState Previous;
    GPIO_PinState Current;
    GPIO_TypeDef *GPIO_Port;
    uint16_t GPIO_Pin;
    void (*ClickedCallback)(void);
} KeyHandle_TypeDef;
```

- 3.将结构体地址传入函数供应用层使用

```
void Key_Init(KeyHandle_TypeDef *Handle);
void Key_Scan(KeyHandle_TypeDef *Handle);
```

按键音效

利用二进制信号量创建蜂鸣器任务，由按键的回调函数给出二进制信号量，蜂鸣器任务取信号鸣叫实现

问题改进：

多次按下按键，部分信号丢失

将原来的二进制信号改为计数信号量

```
void Buzzer_Init(void)
{
    // xBinarySemaphoreForBuzzer = xSemaphoreCreateBinary();
    xBinarySemaphoreForBuzzer = xSemaphoreCreateCounting(5, 0);
}
```

LCD (Liquid Crystal Display) 液晶显示器

1. 复位

由手册得知想要启动复位过程就得给RST引脚施加一个大于10us的负向脉冲，太短的负脉冲会被芯片忽略。并且启动复位过程后不能马上执行后续的操作，至少应当等待125ms保证复位完成

```
// @作用：使用RST引脚对屏幕进行复位
//
static void Reset(void)
{
    // 将RST引脚拉低，施加复位信号
    HAL_GPIO_WritePin(LCD_RST_GPIO_Port, LCD_RST_Pin, GPIO_PIN_RESET);
    // 最小延迟时间1ms > 10us
    vTaskDelay(pdMS_TO_TICKS(1));
    // 将RST引脚拉高，释放复位信号
    HAL_GPIO_WritePin(LCD_RST_GPIO_Port, LCD_RST_Pin, GPIO_PIN_SET);
    // 延迟125ms等待复位完成
    vTaskDelay(pdMS_TO_TICKS(125));
}
```

2. 发送命令与数据

开发板上的屏幕使用的是一种叫做“4线串行接口”的方式。如上图，所谓“4线”指的是3条SPI线（也就是SCK、MOSI和NSS）和一条RS线。其中3条SPI线负责传输实际的内容，而RS线用来在数据和命令之间切换。当RS=0时SPI上传输的内容被视为“命令”；当RS=1时SPI上传输的内容被视为“数据”

```
//
// @作用：通过4线串口向LCD发送命令
// @参数：Cmd - 要发送的命令
//
static void SendCommand(uint8_t Cmd)
{
    extern SPI_HandleTypeDef hspil;

    // 切换到命令模式
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_RESET);
    // 选中从机
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_RESET);
    // 使用SPI发送命令
    HAL_SPI_Transmit(&hspil, &Cmd, 1, HAL_MAX_DELAY);
    // 取消选中
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_SET);
}
//
// @作用：通过4线串口向LCD发送数据
//
static void SendData(uint8_t *pData, uint16_t Size)
```

```

{
    extern SPI_HandleTypeDef hspi1;

    // 切换到数据模式
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_SET);
    // 选中从机
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_RESET);
    // 使用SPI发送数据
    HAL_SPI_Transmit(&hspi1, pData, Size, HAL_MAX_DELAY);
    // 取消选中
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_SET);
}

```

3.显示图像

设置图像显示范围

命令	参数1	参数2	参数3	参数4	描述
0x2a	列起始地址的高8位	列起始地址的低8位	列结束地址的高8位	列结束地址的低8位	设置列范围
0x2b	行起始地址的高8位	行起始地址的低8位	行结束地址的高8位	行结束地址的低8位	设置行范围

产生图像数据

ST7789规定像素的填充顺序按照“自下而上，逐行扫描”。也就是说，当设置好图像的显示范围之后，我们需要从左下角第一个像素点开始，把每一个像素点的数据发送给LCD。

我们使用了一种叫做“16位-65k RGB565”的颜色格式。该格式规定每个像素点占两个字节，也就是16个比特位。在这16个比特位当中红色占5个比特，绿色占6个比特，蓝色占5个比特，也就是RRRR RGGG GGGB BBBB

4.轮询标志位缺点

```

void HAL_SPI_Transmit(SPI_HandleTypeDef *hspi, const uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    // 将数据逐个发送出去
    for(uint16_t i=0; i<Size; i++)
    {
        // 等待TXE标志位位置位（只有发送数据寄存器有空位才能继续写入）
        while(__HAL_SPI_GET_FLAG(hspi, SPI_FLAG_TXE) == 0);

        // 写入要发送的数据
        hspi->Instance->DR = pData[i];
    }

    // 等待最后一个数据发送完成
    while(__HAL_SPI_GET_FLAG(hspi, SPI_FLAG_BSY) == 0);
}

```

该方法持续占用CPU资源，让其他低优先级的任务等不到执行机会

改进方法：二进制信号量+DMA

基本原理是：

1. CPU将搬运数据的工作交给DMA

2. LCD任务在等待DMA搬运完成期间使用二进制信号量进入阻塞

```
//  
// @作用: 通过4线串口向LCD发送数据  
//  
static void SendData(uint8_t *pData, uint16_t Size)  
{  
    extern SPI_HandleTypeDef hspi1;  
  
    // 切换到数据模式  
    HAL_GPIO_WritePin(LCD_RS_GPIO_Port, LCD_RS_Pin, GPIO_PIN_SET);  
    // 选中从机  
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_RESET);  
    // 使用SPI发送数据  
    // HAL_SPI_Transmit(&hspi1, pData, Size, HAL_MAX_DELAY);  
    HAL_SPI_Transmit_DMA(&hspi1, pData, Size);  
  
    xSemaphoreTake(xSemaphoreForDMA, portMAX_DELAY);  
  
    // 取消选中  
    HAL_GPIO_WritePin(LCD_NSS_GPIO_Port, LCD_NSS_Pin, GPIO_PIN_SET);  
}
```

DMA传输完成后给信号让SendData任务推出阻塞

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi)  
{  
    if(hspi->Instance == SPI1)  
    {  
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;  
  
        xSemaphoreGiveFromISR(xSemaphoreForDMA, &xHigherPriorityTaskWoken);  
  
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);  
    }  
}
```

万用表测量功能

2026年9月22日 19:01

1.挡位检测

由开发板原理图得知，开关公共端与3.3V相连，也就是切换到哪个挡位单片机的对应引脚就会直接与3.3V相连，所以将各个开关挡位端口设置为输入下拉

创建dmm.c/.h文件

定义结构体储存挡位信息

```
typedef enum
{
    eDMMRange2V,      // 2V电压挡
    eDMMRange10V,     // 10V电压挡
    eDMMRange50V,     // 50V电压挡
    eDMMRange1kOhm,   // 1k电阻档
    eDMMRange10kOhm,  // 10k电阻档
    eDMMRange100kOhm, // 100k电阻档
    eDMMRange1MOhm,   // 1M电阻档
} DMMRange_t;
```

用枚举读各个端口电平确定挡位

```
static DMMRange_t DMM_GetRange(void)
{
    DMMRange_t range = eDMMRange2V;

    if(HAL_GPIO_ReadPin(DMM50V_GPIO_Port, DMM50V_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange50V; // 50V
    }
    else if(HAL_GPIO_ReadPin(DMM10V_GPIO_Port, DMM10V_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange10V; // 10V
    }
    else if(HAL_GPIO_ReadPin(DMM2V_GPIO_Port, DMM2V_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange2V; // 2V
    }
    else if(HAL_GPIO_ReadPin(DMM10kOhm_GPIO_Port, DMM10kOhm_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange10kOhm; // 10kOhm
    }
    else if(HAL_GPIO_ReadPin(DMM100kOhm_GPIO_Port, DMM100kOhm_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange100kOhm; // 100kOhm
    }
    else if(HAL_GPIO_ReadPin(DMM1MOhm_GPIO_Port, DMM1MOhm_Pin) == GPIO_PIN_SET)
    {
        range = eDMMRange1MOhm; // 1MOhm
    }
    else
    {
        range = eDMMRange1kOhm; // 1kOhm
    }
}
```

```

        return range;
    }

```

2.ADC读取

1. 使能定时器2，它负责产生10Hz的TRGO信号来触发右侧的ADC；
2. ADC1使用注入序列的IN9采集数据，将左侧的TRGO作为触发信号，因此每100ms采集一次；

```

//
// @作用：初始化数字万用表模块
//
void vDMMInit(void)
{
    // 创建邮箱，队列项的尺寸为DMMData_t
    xMailbox = xQueueCreate(1, sizeof(DMMData_t));
    // 启动定时器2，该定时器用于触发ADC1的注入序列
    extern TIM_HandleTypeDef htim2;
    HAL_TIM_Base_Start(&htim2);
    // 对ADC进行校准并启动ADC
    extern ADC_HandleTypeDef hadc1;
    HAL_ADCEX_Calibration_Start(&hadc1);
    HAL_ADCEX_InjectedStart_IT(&hadc1);
}

```

3. ADC1的注入序列转换结束后触发中断，进而调用回调函数HAL_ADCEX_InjectedConvCpltCallback();
4. 在回调函数里完成数据采集，并把采集的结果填入到结构体DMMData_t里；
5. 调用xQueueOverwriteFromISR()将结构体内的数据发送给邮箱；

```

//
// @作用：ADC的注入序列转换结束后会触发此回调函数
//
void HAL_ADCEX_InjectedConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(hadc->Instance == ADC1) // 仅处理ADC1的结果
    {
        // 万用表模块的运算放大电路的输出电压
        float fVdmm_adc = HAL_ADCEX_InjectedGetValue(hadc, ADC_INJECTED_RANK_1) / 4095.0f * 3.3f;

        DMMRange_t eDMMRange = DMM_GetRange(); // 获取当前档位
        float fDMMValue = prvCalcDMMValue(fVdmm_adc, eDMMRange); // 计算结果

        DMMData_t xDmmData;

        xDmmData.eDMMRange = eDMMRange;
        xDmmData.fDMMValue = fDMMValue;

        xQueueOverwriteFromISR(xMailbox, &xDmmData, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

6. 编写一个xDMMGetData()函数，其它任务可以通过这个函数获取万用表的数据，而xDMMGetData()本身是通过xQueuePeek()来读取邮箱数据的；

```

//

```

```

// @作用：根据ADC的结果和万用表的挡位计算最终结果
//
static float prvCalcDMMValue(float fVadc, DMMRange_t eRange)
{
    float result;

    float fVin = fVadc * 2.0f - 2.0f;

    if(eRange == eDMMRange2V) // 电压档2V
    {
        result = fVin * 1.0f;
    }
    else if(eRange == eDMMRange10V) // 电压档10V
    {
        result = fVin * 5.0f;
    }
    else if(eRange == eDMMRange50V) // 电压档50V
    {
        result = fVin * 25.0f;
    }
    else // 以下为电阻档
    {
        float R0, R1, Rx;

        if(eRange == eDMMRange1kOhm) // 电阻档1k
        {
            R0 = 200.0f;
        }
        else if(eRange == eDMMRange10kOhm) // 电阻档10k
        {
            R0 = 2000.0f;
        }
        else if(eRange == eDMMRange100kOhm) // 电阻档100k
        {
            R0 = 2.0e4f;
        }
        else // 电阻档1M
        {
            R0 = 2.0e5f;
        }

        R1 = 2.0e6f;
        Rx = fVin / (2.5 - fVin) * R0;

        result = R1 * Rx / (R1 - Rx);
    }

    return result;
}

//
// @作用：获取数字万用表的测量结果
//
DMMData_t xDMMGetData(void)
{
    DMMData_t xResult;

```

```

// 从邮箱里把数据读出来
xQueuePeek(xMailbox, &xResult, portMAX_DELAY);

return xResult;
}

```

2.稳压电源

在开发板上有一个“稳压电源模块”，它可以输出3.3V~24V可调的电压，是个很方便的工具

为了实时监测稳压模块的输出电压值，DC-DC升压模块的输出经过1/13衰减后被接回到单片机的PB0引脚，PB0引脚对应的是ADC的通道8

采集稳压电源输出的引脚是PB0，对应ADCx_IN8。打开CubeMx图形化配置界面，使能IN8，并将它加入到注入序列的Rank3当中，采样时间取1.5倍周期

打开dmm.h文件，在DMM_Data_t结构体里边增加成员fVpwr，以存储测量到的稳压电源模块的输出电压值

修改dmm.c文件里的ADC中断的回调函数，在其中增加采集稳压电源模块输出电压的代码

```

//
// @作用：ADC的注入序列转换结束后会触发此回调函数
//
void HAL_ADCEx_InjectedConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(hadc->Instance == ADC1) // 仅处理ADC1的结果
    {
        // 采集Vrefint，反推VDDA
        float fVdda = 4095.0f / HAL_ADCEx_InjectedGetValue(hadc, ADC_INJECTED_RANK_2) * 1.205f;
        // 万用表模块的运算放大电路的输出电压
        float fVdmm_adc = HAL_ADCEx_InjectedGetValue(hadc, ADC_INJECTED_RANK_1) / 4095.0f * fVdda;
        // 稳压电源模块的输出电压
        float fVpwr = HAL_ADCEx_InjectedGetValue(hadc, ADC_INJECTED_RANK_3) / 4095.0f * fVdda * 13.0f;

        DMMRange_t eDMMRange = DMM_GetRange(); // 获取当前挡位
        float fDMMValue = prvCalcDMMValue(fVdmm_adc, eDMMRange); // 计算结果

        DMMData_t xDmmData;

        xDmmData.eDMMRange = eDMMRange;
        xDmmData.fDMMValue = fDMMValue;
        xDmmData.fVpwr = fVpwr;

        xQueueOverwriteFromISR(xMailbox, &xDmmData, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

万用表UI

2026年9月22日 19:36

1.打印测试结果

在App文件夹下新建一个GUI文件夹，并将GUI文件夹添加到工程的引用路径当中，然后在GUI文件夹当中新建label_dmm.h和lable_dmm.c

在.h文件里声明一个DMMLabel_Repaint()函数。英文单词Repaint是“重绘”的意思，也就是说这个函数负责更新“数字万用表标签”的显示内容

```
//
// @作用：重绘数字万用表标签
//
void DMMLabel_Repaint(void)
{
    // 绘制矩形背景
    LCD_SetCursor(110, 215); // 设置光标到矩形的左上角点
    LCD_SetBrush(0xcccccc); // 设置矩形的填充颜色
    LCD_FillRect(120, 15); // 填充矩形，宽120像素、高15像素

    // 获取数字万用表的数据
    DMMData_t xDmmData = xDMMGetData();

    // 打印挡位信息
    const char *pccRangeStrs[] = {"DMM--2V:", "DMM-10V:", "DMM-50V:", "DMM--1k:",
    "DMM-10k:", "DMM100k:", "DMM--1M:"};
    LCD_SetCursor(116, 226); // 将光标设置到字符串的左下角
    LCD_SetPen(0x000000); // 设置画笔颜色为黑色

    const char *pcRangeStr = pccRangeStrs[xDmmData.eDMMRange]; // 获取挡位字符串

    LCD_PrintString(pcRangeStr); // 打印挡位信息

    // 确定单位到底是“V”还是“R”
    const char *pcUnit; // 单位字符串

    if(xDmmData.eDMMRange < eDMMRange1kOhm) // 电压档
    {
        pcUnit = "V";
    }
    else // 电阻档
    {
        pcUnit = "R";
    }

    // 打印测量结果
    LCD_PrintFloatSI(xDmmData.fDMMValue, 6, pcUnit);
}
```

2.稳压电源

添加label_pwr.h和label_pwr.c文件，并在label_pwr.h和label_pwr.c里新增PwrLabel_Repaint()方法，并编写如下代码：

```
// @作用：重绘稳压电源标签
void PwrLabel_Repaint(void)
{
    // 绘制矩形背景
    LCD_SetCursor(6, 215); // 设置光标到矩形的左上角点
    LCD_SetBrush(0xcccccc); // 设置矩形的填充颜色
    LCD_FillRect(90, 15); // 填充矩形，宽90像素、高15像素

    // 获取数字万用表的数据
    DMMDData_t xDmmData = xDMMGetData();

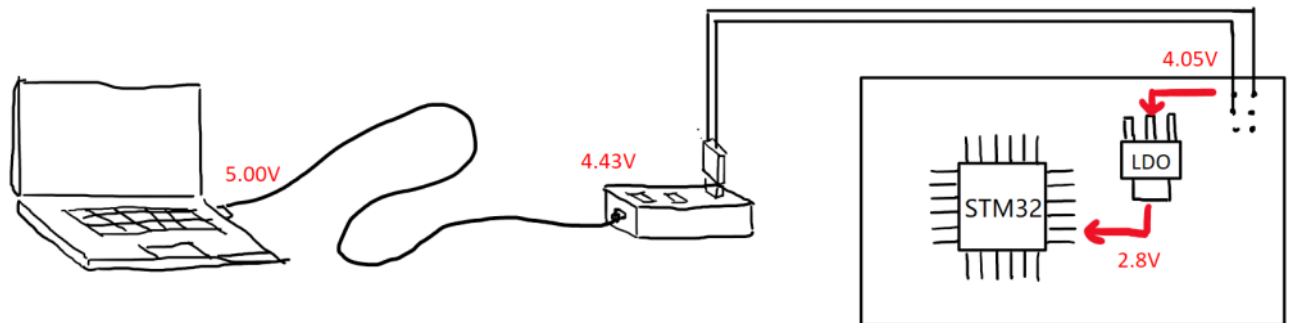
    // 打印测量结果
    LCD_SetPen(0x000000); // 设置黑色画笔
    LCD_SetCursor(18, 226);
    LCD_PrintString("Pwr:");
    LCD_PrintFloatSI(xDmmData.fVpwr, 5, "V");
}
```

万用表问题解决

2026年9月23日 18:51

使用ST-Link供电时，误差非常大，测量结果竟然高达8k欧姆；使用外接USB供电时测量结果恢复正常。这其实是由于ST-LINK供电电压不足导致的。

原因：理想条件下输入到开发板的电压应该是标准的5V，但实际上根本达不到5V。电脑的USB接口虽然能够输出5V的电压，但是经过USB集线器转接后到达开发板的电压就只有4.05V了，而后，4.05V的输入电压要经过LDO芯片降压才能给单片机供电。但是LDO的降落电压要在1.2V以上，因此实际给到单片机的供电电压只剩2.8V，远低于要求的3.3V



解决方案：

ADC模块内部有一个内部参考电压源 - Vref_internal，电压大小为1.2V，我们可以使用它作为参照来校准供电不足造成的误差

打开CubeMx，激活ADC的Vrefint通道，并将它加入到注入序列的Rank2当中，采样时间选择239.5倍周期。采集Vrefint，然后反推出真实的供电电压VDDA，再然后根据真实的VDDA计算万用表测量值。这段代码应该被写在dmm.c的ADC注入序列中断回调函数内部

```
//
// @作用：ADC的注入序列转换结束后会触发此回调函数
//
void HAL_ADCEx_InjectedConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(hadc->Instance == ADC1) // 仅处理ADC1的结果
    {
        // 采集Vrefint，反推VDDA
        float fVdda = 4095.0f / HAL_ADCEx_InjectedGetValue(hadc, ADC_INJECTED_RANK_
2) * 1.205f;
        // 万用表模块的运算放大电路的输出电压
        float fVdmm_adc = HAL_ADCEx_InjectedGetValue(hadc, ADC_INJECTED_RANK_1) /
4095.0f * fVdda;

        DMMRange_t eDMMRange = DMM_GetRange(); // 获取当前挡位
        float fDMMValue = prvCalcDMMValue(fVdmm_adc, eDMMRange); // 计算结果

        DMMData_t xDmmData;

        xDmmData.eDMMRange = eDMMRange;
        xDmmData.fDMMValue = fDMMValue;

        xQueueOverwriteFromISR(xMailbox, &xDmmData, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
```

用户面板

2026年9月23日 19:07

问题：

开发板上有一块LCD显示器，它宽320个像素，高240个像素，每个像素点使用两个字节传输颜色数据。假设每秒钟刷新10次，那么传输数据的波特率至少要达到 $320 \times 240 \times 2 \times 8 \times 10 = 12,288,000\text{bps}$ ，约12Mbps，如此庞大的数据量对于单片机来说是个不小的负担

解决思路：

使用事件组实现分区域的绘制。也就是说我们不会一次性地绘制屏幕上的所有的部件，而是当某个部件的变化时我们才会重新绘制它，这种“按需重绘”的方式能够极大地减少单片机的工作量，也就是在ADC的中断服务程序执行完之后重新绘制标签

具体实现：

在lcd_task.h里为“稳压电源标签”和“数字万用表标签”分配事件组的比特位，分别是bit0和bit1

```
#include "event_groups.h"

#define PWR_LABEL_REPAINT_BIT (EventBits_t)(0x01 << 0) // bit0, 重绘稳压电源标签
#define DMM_LABEL_REPAINT_BIT (EventBits_t)(0x01 << 1) // bit1, 重绘万用表标签
#define ALL_REPAINT_BITS (PWR_LABEL_REPAINT_BIT | DMM_LABEL_REPAINT_BIT) // 所有用到的比特位

改造LCD的任务函数
void vLCDTask(void *pvParameters)
{
    .....

    // 开机后的第一次，所有部件都需要重绘
    xEventGroupSetBits(xEventGroupForLCD, ALL_REPAINT_BITS);

    for(;;)
    {
        EventBits_t xBitRet;

        // xClearOnExit = pdTRUE, 退出时擦除
        // xWaitForAllBits = pdFALSE, 等待任意比特位
        xBitRet = xEventGroupWaitBits(xEventGroupForLCD, ALL_REPAINT_BITS, pdTRUE, pdFALSE, portMAX_DELAY);

        if(xBitRet & PWR_LABEL_REPAINT_BIT)
        {
            PwrLabel_Repaint(); // 重绘稳压电源标签
        }
        if(xBitRet & DMM_LABEL_REPAINT_BIT)
        {
            DMMLabel_Repaint(); // 重绘数字万用表标签
        }
    }
}
```

在ADC的中断服务程序里给对应事件比特位写1

```
void HAL_ADCEx_InjectedConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(hadc->Instance == ADC1) // 仅处理ADC1的结果
    {
        .....
    }
}
```

```

extern EventGroupHandle_t xEventGroupForLCD;

xEventGroupSetBitsFromISR(xEventGroupForLCD,
                          PWR_LABLE_REPAINT_BIT | DMM_LABLE_REPAINT_BIT, // 设置PWR和DMM的重绘事件比特
                          &xHigherPriorityTaskWoken);

portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
}

```

信号发生器面板:

信号发生器面板和所有这些按钮的交互关系。其实它们的交互关系比较简单，可以用三句话来总结：

1. KEY4与信号发生器面板对应，按下KEY4信号发生器面板被选中；再次按下，光标将在参数1和参数2之间切换。
2. KEY1~KEY3与信号发生器面板无关，按下它们当中的任意一个，信号发生器面板将被取消选中，光标将会丢失。
3. 如果信号发生器面板在选中状态，按KEY_UP或KEY_DOWN，将会调节参数的值。

实现“选中”动作 - vGeneratorPanelSelect()

```

//
// @作用：选中信号发生器面板，再次点击时切换光标位置
//
void vGeneratorPanelSelect(void)
{
    // 1. 更改光标的位置
    if(iSelectedIndex < 0) // 未选中状态，按钮按下后光标选中第一个参数
    {
        iSelectedIndex = 0;
    }
    else // 已经选中，光标在参数之间切换
    {
        iSelectedIndex++;
        if(iSelectedIndex > 1)
        {
            iSelectedIndex = 0;
        }
    }

    extern EventGroupHandle_t xEventGroupForLCD;

    // 2. 通过设置事件组的比特位，来触发信号发生器面板重绘
    xEventGroupSetBits(xEventGroupForLCD, GENERATOR_PANEL_REPAINT_BIT);
}

```

在KEY4按钮按下的时候调用刚才写的vGeneratorPanelSelect()

实现vGeneratorPanelDeselect()函数

```

//
// @作用：取消选中信号发生器面板
//
void vGeneratorPanelDeselect(void)
{
    if(iSelectedIndex >= 0)
    {
        // 1. 将变量iSelectedIndex的值设置为负数，表示取消选中
        iSelectedIndex = -1;

        extern EventGroupHandle_t xEventGroupForLCD;

        // 2. 通过设置事件组的比特位，来触发信号发生器面板重绘
        xEventGroupSetBits(xEventGroupForLCD, GENERATOR_PANEL_REPAINT_BIT);
    }
}

```

```

    }
}

```

当按钮KEY1~KEY3按下的时候，调用刚刚写好的vGeneratorPanelDeselect()

最后要实现的动作是“调整”，即调整参数。如下图，信号发生器面板共有两个参数，当光标选中后可以通过KEY_UP和KEY_DOWN。

调整这些参数的值。比如，参数1用来设置波形的形状，当你按下KEY_UP或者KEY_DOWN时，该参数就会在4种波形之间循环切换：OFF、Sin、Triangle和Square。再比如，参数2用来设置波形的频率，按KEY_UP提高输出频率、按KEY_DOWN降低输出频率。

```

//
// @作用：通过KEY_UP和KEY_DOWN按钮调整信号发生器面板的参数
// @参数：iDirection - 调整方向。+1 - 向上，对应KEY_UP，-1 - 向下，对应KEY_DOWN
//
void vGeneratorPanelAdjust(int iDirection)
{
    if(iSelectedIndex < 0) return; // 光标未选中，不做任何操作

    // 1. 如果参数1被选中，更改波形的形状
    if(iSelectedIndex == 0)
    {
        eWaveformShape = (eWaveformShape - iDirection + 4) % 4;
    }

    // 2. 如果参数2被选中，循环更改波形的频率
    if(iSelectedIndex == 1)
    {
        static const float freqs[] = {50.0f, 100.0f, 200.0f, 500.0f, 1.0e3f, 2.0e3f, 5.0e3f, 10.0e3f,
50.0e3f };
        static const int numOfFreqs = sizeof(freqs) / sizeof(float);

        int i = 0;
        // 通过一个for循环，定位到当前频率在数组当中的序号
        for(i=0; i<numOfFreqs; i++)
        {
            if(freqs[i] == fWaveformFrequency)
            {
                break;
            }
        }
        i = i+iDirection; // 计算下一个频率的值
        if(i>numOfFreqs) i = numOfFreqs-1;
        if(i<0) i=0;
        fWaveformFrequency = freqs[i];
    }

    // 3. 通过设置事件组的比特位，来触发信号发生器面板重绘
    extern EventGroupHandle_t xEventGroupForLCD;
    xEventGroupSetBits(xEventGroupForLCD, GENERATOR_PANEL_REPAINT_BIT);
}

```

把这个函数放到KEY_UP和KEY_DOWN两个按钮的回调函数当中调用

运行模式面板：

同上

缩放面板：

缩放（Scale）面板位于LCD屏幕左侧，运行模式（Run Mode）面板之下。面板内部有两个参数：x和y，分别用来调整X轴的缩放比和Y轴的缩放比例。

接下来我们来介绍缩放比例的概念。在LCD屏幕的中央区域是一张波形图，用来显示示波器采集到的波形。我们假设

横轴是X，表示时间，单位是秒；纵轴是Y，表示电压大小。X轴缩放比，XScale，就是每一格代表的时间长短，单位：秒/格；Y轴缩放比，YScale，就是每一格代表的电压大小，单位：伏/格。我们通过调节XScale和YScale的大小就可以实现波形的缩放。

缩放标签：

它显示了两种数值，上边的数值表示X轴向的缩放比，下边的数值表示Y轴向的缩放比。因此当你通过“缩放面板”调节缩放比例的时候就能在黄色标签里观察到它们的变化

游标面板：

有4个参数：x1、x2、y1和y2，它们分别用来移动波形图里的4个游标。为了方便测量波形的数据，我们在波形图里边设置了4个游标，游标x1和游标x2用来测量时间值，游标y1和游标y2用来测量电压值

示波器面板：

思路同上

游标标签：

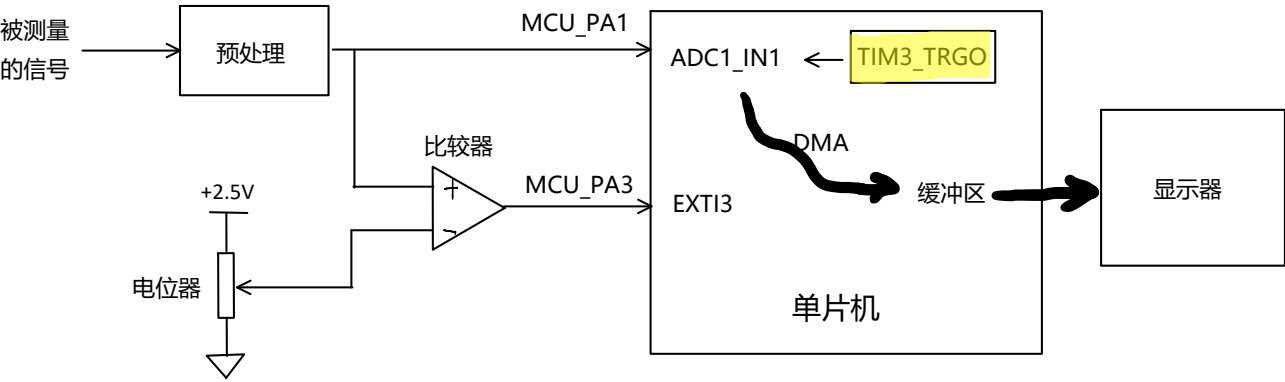
思路同上

触发电压显示：

同上

示波器

2026年9月24日 16:30



1. 被测量的信号。要测量的信号通过这个位置输入到示波器当中。
2. 预处理电路。它的作用是对信号进行预处理，比如把输入信号的幅度调整到0~3.3V范围内，让它能够被单片机的ADC测量。输入信号经过预处理后接入到单片机的ADC当中（PA1，ADC_IN1）。
3. 电位器。用于产生触发电压，用户可以通过调节这个电位器来改变触发电压的大小。
4. 比较器。用于比较被测信号和触发电压的大小，从而产生触发信号。该触发信号会接入到单片机的EXTI3，通过它产生中断，进而在中断服务程序当中启动数据采集。
5. TIM3_TRGO。定时器3的TRGO用于产生触发信号，从而驱动ADC1的常规序列工作。
6. DMA。DMA用于在ADC和缓冲区之间搬运数据。数据经过处理后显示到LCD屏上。

在App文件夹下创建waveform_capture.h和waveform_capture.c文件，英文单词Waveform capture是波形捕捉器的意思。

创建WaveformCapture_Init()方法，用于波形捕捉器的初始化，在该函数内部开启TIM3（用于产生ADC的外部触发信号）。

waveform_capture.h

```
void WaveformCapture_Init(void); // 初始化波形捕捉器
```

waveform_capture.c

```
// @作用：初始化波形捕捉器
```

```
void WaveformCapture_Init(void)
{
    extern TIM_HandleTypeDef htim3;

    // 启动定时器3，该定时器用于ADC1常规序列的外部触发
    HAL_TIM_Base_Start(&htim3);
}
```

main.c

```
int main(void)
{
    WaveformCapture_Init(); // 在main方法里调用该函数
}
```

编写EXTI3中断的回调函数，在回调函数里以DMA方式启动ADC。注意，我们应当声明一个数组rawDataBuffer来保存采集到的原始数据。

waveform_capture.c

```
static uint16_t rawDataBuffer[1024]; // 用于缓存ADC的原始数据

// EXTI中断的回调函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    extern ADC_HandleTypeDef hadc1;
```

```

    if(GPIO_Pin == Trigger_Pin)
    {
        // 以DMA方式启动ADC
        HAL_ADC_Start_DMA(&hadc1, (uint32_t *)rawDataBuffer, 1024);
    }
}

```

ADC数据采集完成后会由DMA触发中断，该中断对应的回调函数为HAL_ADC_ConvCpltCallback()。我们需要在回调函数里对采集到的数据进行处理，我们将处理后的波形数据保存到waveform数组当中

waveform_capture.c

```

static float waveform[1024]; // 声明一个数组，用来保存处理后的数据

// ADC常规序列数据采集完成的回调函数
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        // 1. 对采集到的原始数据进行处理
        for(int i=0; i<1024; i++)
        {
            waveform[i] = (rawDataBuffer[i] / 4095.0f * 3.3f) * 2.0f - 2.5f;
        }

        // 2. 通知LCD任务重绘波形面板
        extern EventGroupHandle_t xEventGroupForLCD;
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        xEventGroupSetBitsFromISR(xEventGroupForLCD, WAVEFORM_PANEL_REPAINT_BIT, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

把波形数据“交给”波形面板，让它显示出来

waveform_capture.h

```

float *WaveformCapture_GetWaveform(void); // 获取波形数组

```

waveform_capture.c

```

// @作用：获取波形数组
float *WaveformCapture_GetWaveform(void)
{
    return waveform;
}

```

waveform_panel.c

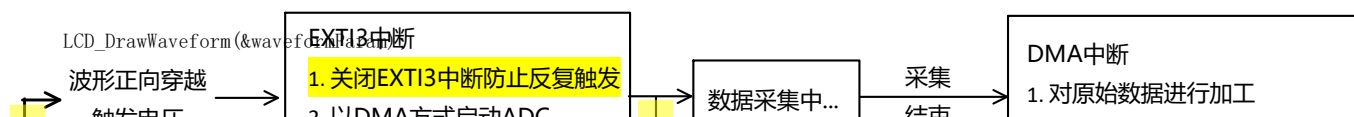
```

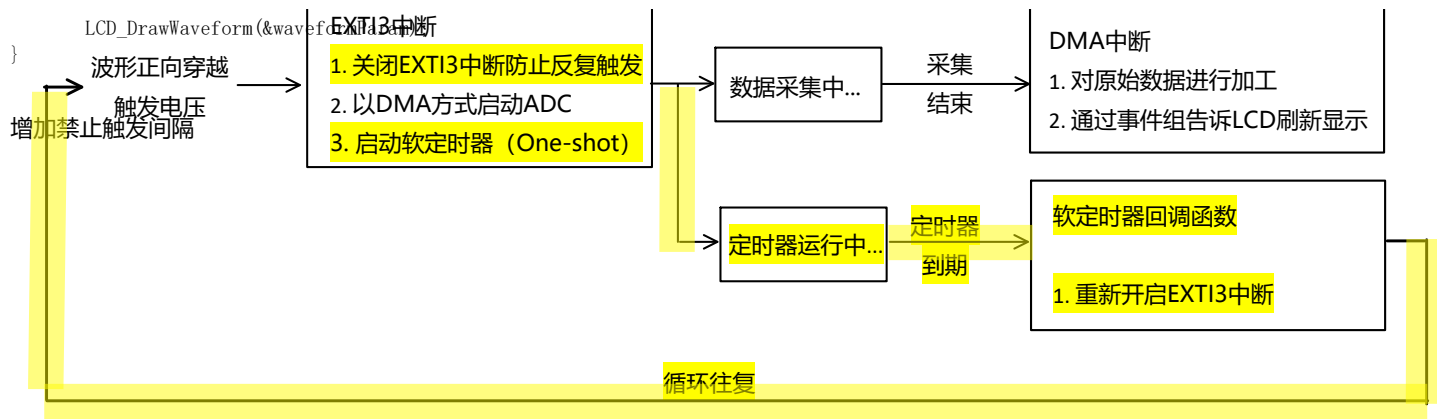
#include "waveform_capture.h"

// 重绘波形面板
void RepaintWaveformPanel(void)
{
    WaveformParamTypeDef waveformParam = {0};

    waveformParam.Waveform = WaveformCapture_GetWaveform();
    waveformParam.WaveLength = 1024;
    waveformParam.SampleRate = 10e3; // 先固定10k采样率
    waveformParam.XScale = fGetXScale();
    waveformParam.YScale = fGetYScale();
    waveformParam.CursorX1Pct = GetCursorX1Pct();
    waveformParam.CursorX2Pct = GetCursorX2Pct();
    waveformParam.CursorY1Pct = GetCursorY1Pct();
    waveformParam.CursorY2Pct = GetCursorY2Pct();
}

```





如图，为了实现禁止触发间隔我们使用了软件定时器（黄色为新增部分），可以看到我们新增的动作有以下几个：

1. 为了避免重复触发，在EXTI3中断发生后立即关闭中断
2. 启动软件定时器开始计时
3. 软件定时器到期后重新打开EXTI3中断

这样就可以实现软件定时器运行期间一直保持EXTI3中断关闭，从而杜绝重复触发，禁止触发的间隔由软件定时器的Period参数设置

在波形捕捉器的初始化函数WaveformCapture_Init()当中创建一个软件定时器，名为xHoldoffTimer。这个定时器用来实现禁止触发间隔，周期为10ms，不自动重装

waveform_capture.c

```
static TimerHandle_t xHoldOffTimer; // 软件定时器，用于禁止触发

// @作用：初始化波形捕捉器
void WaveformCapture_Init(void)
{
    .....

    // 创建软件定时器，用于禁止触发（10ms）
    xHoldOffTimer = xTimerCreate("Hold Off", pdMS_TO_TICKS(10), pdFALSE, NULL, HoldOffTimerCallback);
}
```

当EXTI3中断发生时，立刻屏蔽该中断，并且启动xHoldOffTimer定时器。

waveform_capture.c

```
// EXTI中断的回调函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    extern ADC_HandleTypeDef hadc1;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(GPIO_Pin == Trigger_Pin)
    {
        HAL_ADC_Start_DMA(&hadc1, (uint32_t *)rawDataBuffer, 1024); // 以DMA方式启动ADC

        HAL_NVIC_DisableIRQ(Trigger_EXTI_IRQn); // 屏蔽中断，以防再次触发

        // 启动软件定时器，到期后再恢复中断
        xTimerStartFromISR(xHoldOffTimer, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
```

编写定时器的回调函数，在该函数内部恢复EXTI3的中断。注意！恢复中断之前先要清除之前挂起的中断标志，否则会误触发。

waveform_capture.c

```
// 禁止触发定时器的回调函数
```

```
static void HoldOffTimerCallback(TimerHandle_t xTimer)
{
    // 清除中断标志
    __HAL_GPIO_EXTI_CLEAR_FLAG(GPIO_PIN_3);
    HAL_NVIC_ClearPendingIRQ(EXTI3_IRQn);

    // 再次开启中断, 禁止触发结束
    HAL_NVIC_EnableIRQ(EXTI3_IRQn);
}
```

示波器问题1解决

2026年9月24日 18:37

之前的触发机制，能够让示波器能够每次在相同位置开始数据采集，从而在屏幕上显示清晰稳定的图像。但它也有一个致命的问题，如果波形落到了触发电压以外，始终不能满足触发条件，那么我们就看不到这个波形了

解决思路：设置一个期限，如果太长时间没有正常触发就强制触发一次

在波形捕捉器的初始化函数里创建一个用于强制触发的软件定时器，xForceTriggerTimer，其周期选择100ms（这个值等于强制触发的时间间隔），自动重装。创建完成后启动定时器

waveform_capture.c

```
static TimerHandle_t xForceTriggerTimer; // 软件定时器，用于强制触发

// @作用：初始化波形捕捉器
void WaveformCapture_Init(void)
{
    .....

    // 创建软件定时器，用于强制触发
    xForceTriggerTimer = xTimerCreate("Force Trigger", pdMS_TO_TICKS(100), pdTRUE, NULL, ForceTriggerTimerCallback);

    // 启动该定时器
    xTimerStart(xForceTriggerTimer, portMAX_DELAY);
}
```

在EXTI3的回调函数里使用xTimerResetFromISR()复位定时器。

waveform_capture.c

```
// EXTI中断的回调函数
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    extern ADC_HandleTypeDef hadc1;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if(GPIO_Pin == Trigger_Pin)
    {
        // 以DMA方式启动ADC
        HAL_ADC_Start_DMA(&hadc1, (uint32_t *)rawDataBuffer, 1024);

        // 屏蔽中断，以防再次触发
        HAL_NVIC_DisableIRQ(Trigger_EXTI_IRQn);

        // 启动软件定时器，到期后再恢复中断
        xTimerStartFromISR(xHoldOffTimer, &xHigherPriorityTaskWoken);

        // 复位强制触发定时器
        xTimerResetFromISR(xForceTriggerTimer, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
```

```
}
```

在软件定时器的回调函数里手动触发EXTI3中断。

waveform_capture.c

```
// 强制触发定时器的回调函数
```

```
static void ForceTriggerTimerCallback(TimerHandle_t xTimer)
```

```
{
```

```
    __HAL_GPIO_EXTI_GENERATE_SWIT(GPIO_PIN_3);
```

```
}
```

示波器问题2解决

2026年9月25日 11:35

仍有一个致命的问题没有解决，那就是采样率的问题。所谓采样率，就是ADC每秒从波形里采集多少个点出来，其实也就是采样点的密度。采样率选的不合适，将直接拉低示波器的显示效果。示波器使用STM32的ADC模块采集波形，而ADC模块的触发信号又是由TIM3产生的，因此我们只要调节TIM3_TRGO的频率即可调整ADC的采样率。具体来说就是调整TIM3的PSC（预分频器）和ARR（自动重装寄存器）的值

waveform_capture.h

```
void WaveformCapture_SetSampleRate(uint16_t Psc, uint16_t Arr); // 设置波形捕捉器的采样率
uint32_t WaveformCapture_GetSampleRate(void); // 获取波形捕捉器的采样率
```

waveform_capture.c

```
// @作用：设置波形捕捉器的采样率
// @参数：Psc - 预分频器的值
// @参数：Arr - 自动重装寄存器的值
void WaveformCapture_SetSampleRate(uint16_t Psc, uint16_t Arr)
{
    extern TIM_HandleTypeDef htim3;

    HAL_TIM_Base_Stop(&htim3); // 零食关闭定时器

    __HAL_TIM_SET_PRESCALER(&htim3, Psc); // 设置PSC的值
    __HAL_TIM_SET_AUTORELOAD(&htim3, Arr); // 设置ARR的值

    HAL_TIM_Base_Start(&htim3); // 恢复定时器
}

// @作用：获取波形捕捉器的采样率
uint32_t WaveformCapture_GetSampleRate(void)
{
    extern TIM_HandleTypeDef htim3;

    uint16_t psc = htim3.Instance->PSC; // 获取PSC的值
    uint16_t arr = htim3.Instance->ARR; // 获取ARR的值

    // 采样率 = 72MHz / (psc+1) / (arr+1)
    return (uint32_t) (72.0e6f / (psc+1) / (arr+1));
}
```

在过去的程序里我们使用一个1024点的缓冲区存储采集到的波形，那么每次采样的点数就是1024点，它是固定不变的。要想达到最清晰、最完整的显示效果，我们应该把这1024个点铺满屏幕。根据这个原则，采样率可以用如下公式计算：

$$\text{最优采样率} = \frac{\text{采样点数}}{\text{单屏时间跨度}} = \frac{1024\text{点}}{\text{xScale} * 8\text{格}}$$

为了保险起见，我们最好采用一半的采样率，也就是在上边公式的基础上乘以0.5。

由公式可知，采样率的大小并非一成不变，它和XScale的大小相关，因此当XScale的值改变的时候我们要重新计算采样率。那XScale什么时候改变呢？答案很显然，它会在我们通过屏幕右侧的缩放面板调整时改变。因此，我们需要在scale_panel.c文件里添加代码来适应这一变化。

查阅之前的代码你可以发现，x轴像的缩放比共分12挡，我们需要精确计算出每个挡位对应的PSC和ARR的值。这里我用表格的形式呈现给大家：

	xScale	最优采样率	PSC	ARR	实际采样率	误差
1	10us	500kHz	0	143	500kHz	0%
2	20us	500kHz	0	143	500kHz	0%
3	50us	500kHz	0	143	500kHz	0%
4	100us	500kHz	0	143	500kHz	0%
5	200us	320kHz	0	225	320kHz	0%
6	500us	128kHz	0	561	128.11kHz	0.086%
7	1ms	64kHz	0	1124	64kHz	0%
8	2ms	32kHz	0	2251	32kHz	0%
9	5ms	12.8kHz	0	5624	12.8kHz	0%
10	10ms	6.4kHz	0	11249	6.4kHz	0%
11	20ms	3.2kHz	0	22549	3.2kHz	0%
12	50ms	1.28kHz	0	56249	1.28kHz	0%

我们根据这张表格来设置PSC和ARR的值。

scale_panel.c

```
//
// @作用：通过KEY_UP和KEY_DOWN按钮调整信号发生器面板的参数
// @参数：iDirection - 调整方向。+1 - 向上，对应KEY_UP，-1 - 向下，对应KEY_DOWN
//
void vScalePanelAdjust(int iDirection)
{
    if(iSelectedIndex < 0) return; // 光标未选中，不做任何操作

    // 1. 如果参数1被选中 XScale
    if(iSelectedIndex == 0)
    {
        xScaleIndex += iDirection;

        if(xScaleIndex >= numOfXScales) xScaleIndex = numOfXScales - 1;
        if(xScaleIndex < 0) xScaleIndex = 0;

        static const uint16_t PSCs[] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
        static const uint16_t ARRr[] = {143, 143, 143, 143, 225, 561, 1124, 2251, 5624,
```

```
11249, 22549, 56249};
```

```
// 根据xScale的值设置采样率
```

```
WaveformCapture_SetSampleRate(PSCs[xScaleIndex], ARR[xScaleIndex]);
```

```
}
```

```
.....
```

```
}
```

使用互斥锁保护波形缓冲区

2026年9月25日 11:43

在示波器工作期间，ADC会把采集到的数据存放在缓冲区waveform[]当中，同时LCD任务也会从waveform[]数组里取数据，然后显示到屏幕上。注意，既然这两个过程都要访问waveform[]，那么它就是一个共享的资源，当两边的访问同时进行时就会产生竞态条件。因此我们要在缓冲区上加一把互斥锁来杜绝竞态条件的发生

在开始使用互斥锁之前我们要先创建它。接下来我们需要在waveform_capture.c文件里创建一个名为xMutexForWaveform的互斥锁，代码如下：

waveform_capture.c

```
SemaphoreHandle_t xMutexForWaveform; // 用户保护波形缓冲区的互斥锁
```

```
// @作用：初始化波形捕捉器
```

```
void WaveformCapture_Init(void)
```

```
{
```

```
.....
```

```
    // 创建用户保护波形缓冲区的互斥锁
```

```
    xMutexForWaveform = xSemaphoreCreateMutex();
```

```
.....
```

```
}
```

waveform[]数组会分别被LCD任务和ADC中断所访问，因此两边都要加锁。LCD任务这一边不涉及中断，代码比较简单，因此我们先给它上锁。上锁的代码要直接写在访问waveform[]数组的位置，且用完后就要归还锁，代码如下：

waveform_panel.c

```
// 重绘波形面板
```

```
void RepaintWaveformPanel(void)
```

```
{
```

```
    WaveformParamTypeDef waveformParam = {0};
```

```
.....
```

```
    extern SemaphoreHandle_t xMutexForWaveform;
```

```
    xSemaphoreTake(xMutexForWaveform, portMAX_DELAY); // 画图之前获取锁
```

```
    LCD_DrawWaveform(&waveformParam);
```

```
    xSemaphoreGive(xMutexForWaveform); // 画图之后归还锁
```

```
}
```

对于ADC侧来说，对波形数据的访问发生在中断当中，即ADC常规序列采集完成的回调函数里，因此加锁的时候需要使用FromISR版本的API。代码如下：

waveform_capture.c

```

// ADC常规序列数据采集完成的回调函数
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        // 获取锁
        if(xSemaphoreTakeFromISR(xMutexForWaveform, &xHigherPriorityTaskWoken) ==
pdPASS)
        {
            // 1. 对采集到的原始数据进行处理
            for(int i=0; i<1024; i++)
            {
                waveform[i] = (rawDataBuffer[i] / 4095.0f * 3.3f) * 2.0f -
2.5f;
            }

            xSemaphoreGiveFromISR(xMutexForWaveform, &xHigherPriorityTaskWoken);

            // 归还锁

            // 2. 通知LCD任务重绘波形面板
            .....
        }

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

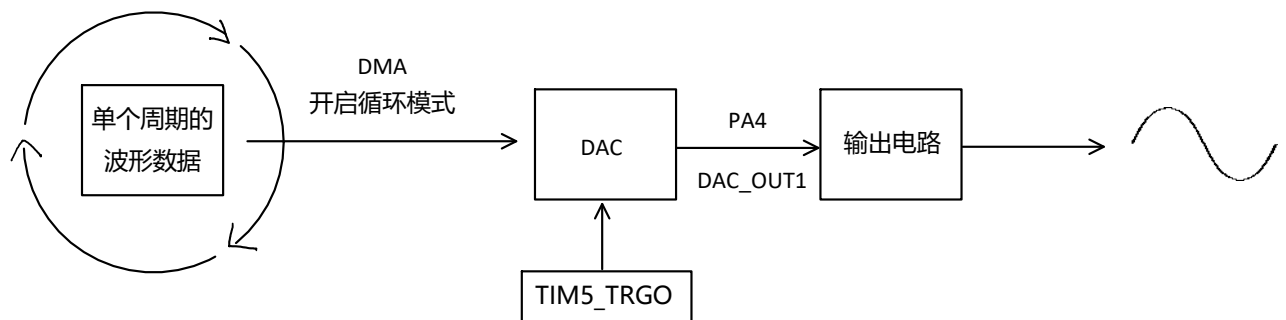
```

信号发生器

2026年9月25日 11:54



信号发生器的基本工作原理如上图，它利用单片机里自带的DAC模块把提前准备好的波形数据（一般存储在一个数组当中）转换为实实在在的波形。实际上，我们开发板里的信号发生器也是这样工作的，但是细节方面需要着重说明。因此我们把这个过程细化一下，就得到了下边的这样一张框图。



- **波形数据**：单个周期的波形数据存放在 `waveform` 数组里，数组中只保存一个周期就够了。
- **DMA**：处于循环模式，循环往复地把这一个周期的数据送入 DAC，从而形成周期信号。
- **DAC**：核心部件，把数字量转换成实实在在的电压。
- **TIM5_TRGO**：定时器 5 的 TRGO 用作 DAC 的外部触发，**每来一个脉冲 DMA 就送一个采样点进 DAC，所以 DAC 的采样率由它决定。**
- **输出电路**：由运放和阻容组成，把 DAC 输出的 $0\sim 3.3\text{V}$ 中实际使用的 $0\sim 2\text{V}$ 搬到 $-2.0\text{V}\sim +2.0\text{V}$ ，同时兼具幅度调节和滤波。

第 2 步：算采样率

采样率 = DAC 每秒向外输出多少个点。原则是让 1000 个点均匀铺满一个波形周期，所以**理想采样率 = 波形频率 $\times$ 1000**。但 DAC 最快只有 500kHz，超了就必须限幅，限幅后点数也要重算。

```
static uint16_t waveform[1000]; // 单个周期的波形数据
static uint16_t waveformLength = 1000; // 波形的实际长度
static uint8_t shape = 0; // 0-OFF, 1-正弦波, 2-三角波, 3-方波
static uint32_t frequency = 1.0e3; // 波形频率, 单位Hz
// @作用：根据波形频率自动调整采样率
sampleRate = 1000 * frequency; // 理想采样率 = 频率  $\times$  点数
if(sampleRate > 500e3) // 超过DAC上限就限幅
    sampleRate = 500e3;
waveformLength = sampleRate / frequency; // 重新计算实际用多少个点
// 根据采样率反推TIM5参数：PSC恒为0, ARR = 72MHz  $\div$  采样率 - 1
__HAL_TIM_SET_PRESCALER(&htim5, psc);
__HAL_TIM_SET_AUTORELOAD(&htim5, arr); // 将PSC和ARR设置到硬件当中
```

以 1kHz 为例：理想需要 1MHz 采样率、1000 个点；限幅到 500kHz 后只用到 **500 个点**，剩下一半用不上。

第 3 步：填充波形数据

先弄清电压换算。输出电路的转换关系是 $V_{out} = (V_{dac} - 1) \times 2$ ，DAC 只用到 $0\sim 3.3\text{V}$ 中的 $0\sim 2\text{V}$ ，所以两个锚点值是：

DAC 电压	DAC 数值	最终输出
1.0V	$4095 \div 3.3 \times 1.0 \approx 1241$	0V

2.0V $4095 \div 3.3 \times 2.0 \approx 2482$ +2V

代码里统一用“把电压转换成 0~4095 的数字”这一步，再按四种波形分别填充：

// 将电压转换为0~4095之间的数字

```
if(shape == 0)           // OFF挡，输出0V电压 → DAC端固定1.0V(1241)
else if(shape == 1)      // Sin挡，输出正弦波 → DAC端0~2V之间摆动
else if(shape == 2)      // Triangle挡，输出三角波 → 线性上升/下降
else if(shape == 3)      // Square挡，输出方波 → 前半段2.0V(2482)、后半段1.0V(1241)
```

四种波形全部封装在 PrepareWaveformData() 里。

第 4 步：加载到 DAC

写一个 RestartDAC()，它是“准备”到“真正输出”的最后一环，做三件事：

// @作用：将波形数据发送到DAC，并重启DAC

```
static void RestartDAC(void)
{
    // 先停止定时器和DAC
    AutoSampleRate();           // 1. 根据波形频率重新计算采样率
    PrepareWaveformData();      // 2. 计算波形数据并填充到waveform数组
    // 开启TIM5
    // 开启DAC (把waveform数组送给DAC并启动)
}
```